

LAB REPORT

For the Course

EC-200 Data Structures



```
C:\MASM611\WORK>lab01
Name: Jawad Ahmed
Degree: 46
Department: Computer Engineering
C:\MASM611\WORK>
```

Group Members	
Jawad Ahmed	
Degree	46
CMS	533047
Syndicate	A
Submitted to	Lab Engr. Hira Irshad

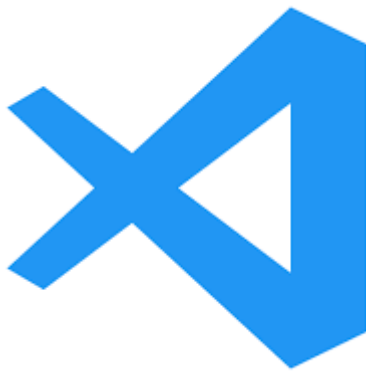
DEPARTMENT OF COMPUTER ENGINEERING
NUST College of Electrical & Mechanical Engineering (CEME)

Lab 10

❖ Software

Software Used: Visual Studio Code.

Usage: The integrated development environment (IDE) was used to write, compile, and debug C++ code. The C++ compiler (MinGW/GCC) was utilized to manage dynamic memory allocation via new and delete keywords, allowing for the creation and destruction of nodes during runtime.



❖ Tasks

```
#include <iostream>
#include <unordered_map>
#include <list>
#include <string>
using namespace std;

struct TrieNode {
    unordered_map<char, TrieNode*> m;
    bool flag;

    TrieNode() : flag(false) {}
};

class Trie {
    TrieNode* root;

public:
    Trie() {
        root = new TrieNode();
    }

    void insertWord(string word) {
        TrieNode* cur = root;
```

```

    for (char c : word) {
        if (cur->m.find(c) == cur->m.end()) {
            cur->m[c] = new TrieNode();
        }
        cur = cur->m[c];
    }
    cur->flag = true;
}

bool searchPrefix(string word) {
    TrieNode* cur = root;

    for (char c : word) {
        if (cur->m.find(c) == cur->m.end()) {
            return false;
        }
        cur = cur->m[c];
    }
    return true;
}

bool searchWord(string word) {
    TrieNode* cur = root;

    for (char c : word) {
        if (cur->m.find(c) == cur->m.end()) {
            return false;
        }
        cur = cur->m[c];
    }
    return cur->flag;
}

void deleteWord(string word) {
    deleteHelper(root, word, 0);
}

void retrieveAllWords(list<string>& allWords) {
    string current = "";
    retrieveHelper(root, current, allWords);
}

private:
bool deleteHelper(TrieNode* node, const string& word, int depth) {
    if (!node) { return false; }

    if (depth == (int)word.length()) {
        if (!node->flag) { return false; }
        node->flag = false;
        return node->m.empty();
    }

    char c = word[depth];
    if (node->m.find(c) == node->m.end()) { return false; }

    bool shouldDelete = deleteHelper(node->m[c], word, depth + 1);

    if (shouldDelete) {
        delete node->m[c];
    }
}

```

```

        node->m.erase(c);
        return !node->flag && node->m.empty();
    }
    return false;
}

void retrieveHelper(TrieNode* node, string& current, list<string>& allWords) {
    if (!node) { return; }

    if (node->flag) {
        allWords.push_back(current);
    }

    for (auto pair : node->m) {
        current += pair.first;
        retrieveHelper(pair.second, current, allWords);
        current.pop_back();
    }
}

};

int main() {
    Trie t;
    t.insertWord("apple");
    t.insertWord("app");
    t.insertWord("bat");
    t.insertWord("ball");
    t.insertWord("banana");

    cout << "Prefix 'app' exists? " << t.searchPrefix("app") << endl; // 1
    cout << "Prefix 'ba' exists? " << t.searchPrefix("ba") << endl; // 1
    cout << "Prefix 'cat' exists? " << t.searchPrefix("cat") << endl; // 0

    cout << "Word 'apple' exists? " << t.searchWord("apple") << endl; // 1
    cout << "Word 'appl' exists? " << t.searchWord("appl") << endl; // 0
    cout << "Word 'banana' exists? " << t.searchWord("banana") << endl; // 1

    t.deleteWord("app");
    cout << "After deleting 'app', search 'app': " << t.searchWord("app") << endl; // 0
    cout << "After deleting 'app', search 'apple': " << t.searchWord("apple") << endl; // 1

    list<string> allWords;
    t.retrieveAllWords(allWords);
    cout << "All words in trie:" << endl;
    for (string w : allWords) {
        cout << w << endl;
    }

    return 0;
}

```

OUTPUT:

```
Prefix 'app' exists? 1
Prefix 'ba' exists? 1
Prefix 'cat' exists? 0
Word 'apple' exists? 1
Word 'appl' exists? 0
Word 'banana' exists? 1
After deleting 'app', search 'app': 0
After deleting 'app', search 'apple': 1
All words in trie:
banana
ball
bat
apple
```